

Simulation-Grouped Per-Class QoS Prediction and SLA-Aware Bandwidth Allocation for SD-WAN

Radhika (Rishiv) Shitlani
School of Computer Science
University of Galway, Galway, Ireland

Abstract—Software-defined wide area networks (SD-WANs) require class-aware quality-of-service (QoS) decisions, yet a strong global prediction score can conceal poor performance for best-effort traffic. This paper studies per-flow delay prediction and service-level-agreement (SLA) violation detection using 29,280 flow records from 400 BNNetSimulator runs spanning Gold, Silver, and Bronze traffic under three scheduling policies. An XGBoost regressor is evaluated with five-fold cross-validation grouped by simulation identifier, ensuring that all flows from a simulator run remain in one fold. The grouped protocol achieves an overall log-delay R^2 of 0.747; class-specific R^2 values are 0.907, 0.898, and 0.680 for Gold, Silver, and Bronze, respectively. Corresponding SLA-violation F1 scores are 0.940, 0.680, and 0.554. A row-wise split gives a higher overall R^2 of 0.766, quantifying the optimism caused by sharing simulation contexts across folds. Under the grouped protocol, 90th-percentile delay is only moderately predictable ($R^2 = 0.293$ overall) and jitter is not predictable ($R^2 = -0.040$). Finally, a model-assisted weighted fair queueing recommender selects 60/30/10 for Gold/Silver/Bronze under the tested mean-delay thresholds. The results show that grouped, per-class evaluation is necessary for credible QoS prediction and bounded allocation support.

Index Terms—SD-WAN, quality of service, machine learning, XGBoost, grouped cross-validation, SLA prediction, bandwidth allocation

I. INTRODUCTION

Software-defined wide area networking (SD-WAN) separates network control from forwarding and enables centrally managed routing and QoS policies across heterogeneous links. In practice, however, many policies remain static or reactive: an operator changes queue weights or routes after congestion has already affected users. Predicting QoS before an SLA breach could enable proactive control, but operational usefulness depends on more than one aggregate regression score. Gold, Silver, and Bronze traffic have different priorities, delay distributions, and costs of error. A model that performs well globally can still be unreliable for the class most exposed to congestion.

Prior work has used machine learning for virtual-network-function scaling [1], application-aware routing [2], and reinforcement-learning-based traffic engineering [3], [4]. Other studies optimise SD-WAN routes and queueing policies from measured QoS [5], [6]. These approaches demonstrate the value of predictive or adaptive control, but they do not directly establish how accurately continuous delay and SLA violations can be predicted for differentiated traffic classes using a compact, reproducible feature set.

A second concern is evaluation leakage. A packet-level simulation produces many flow rows from the same network run. Randomly splitting individual rows can place flows from one simulation in both training and validation data. Although the simulation identifier is not an input feature, shared topology, load, and scheduler conditions may still make the validation task artificially familiar. This paper therefore treats the simulation run, rather than the flow row, as the resampling unit.

The contributions are threefold:

- a reproducible per-class delay-prediction study using 29,280 flows from 400 BNNetSimulator runs;
- a simulation-grouped evaluation reporting both regression and SLA-trigger performance separately for Gold, Silver, and Bronze traffic, with row-wise cross-validation retained only as a sensitivity comparison; and
- an offline, model-assisted weighted fair queueing (WFQ) recommender that ranks candidate allocations under class-dependent mean-delay thresholds.

II. RELATED WORK

Machine learning has been applied to several adjacent network-management tasks. Rahman *et al.* predicted scaling requirements for virtual network functions [1], while Zheng *et al.* combined traffic classification with QoS-aware routing [2]. For SD-WAN specifically, Quang *et al.* formulated joint routing and QoS policy optimisation [5] and later incorporated passive available-bandwidth estimation into a two-loop controller [6]. Recent work also explores safe deep reinforcement learning [3] and multi-objective graph-based traffic engineering [4].

Network simulators and learned network models provide a reproducible alternative when labelled enterprise telemetry is unavailable. BNNetSimulator generates packet-level data compatible with the modelling methodology underlying RouteNet-Fermi [7], [8]. The present study uses it for a narrower question: whether static configuration and path features support reliable, class-specific QoS prediction and a simple allocation decision. XGBoost is selected as the main model because boosted trees remain strong on small-to-medium tabular datasets [9], [10], while retaining comparatively low training cost and feature-level interpretability.

III. METHODOLOGY

A. Dataset and QoS Classes

The dataset was generated with the official BNNetSimulator container [7]. It contains 29,280 source-destination flow records from 400 simulations, with 30–132 flows per run. Traffic is divided into Gold (4,592 rows), Silver (8,991), and Bronze (15,697). Gold has the highest priority, Silver represents interactive traffic, and Bronze is best effort. The experiments cover strict priority, WFQ, and deficit round robin scheduling within four scenarios: fixed WFQ, random WFQ profiles, mixed policies with skewed class proportions, and mixed policies with equal proportions. Loads are light to moderate; consequently, the results should not be interpreted as a stress-test of a saturated production network.

Each input row contains traffic class, offered bandwidth, traffic distribution, route length, network size, maximum average load, link bandwidth, scheduling policy, class queue weight, and minimum class weight. The target is the natural logarithm of average delay in seconds. Delay, jitter, packet loss, delay percentiles, achieved bandwidth, scenario label, and simulation identifier are excluded from the feature matrix. Numeric missing values are median-imputed and categorical variables are one-hot encoded inside each training fold.

B. Prediction and Grouped Evaluation

The primary regressor is XGBoost with 200 trees, depth four, learning rate 0.05, row subsampling 0.9, and feature subsampling 0.9. Predictions are converted back to milliseconds for interpretation. Performance is measured using mean absolute error (MAE) on the delay scale and R^2 on log delay.

Five-fold GroupKFold cross-validation is used with *simulation_id* as the group. Thus, no simulation contributes flows to both training and validation in a fold. For sensitivity analysis, the same model is also evaluated using shuffled row-wise five-fold cross-validation. Preprocessing and model fitting occur independently inside every fold.

Continuous delay predictions are converted into SLA-violation triggers using thresholds of 30, 50, and 60 ms for Gold, Silver, and Bronze. Precision measures the trustworthiness of a trigger, while recall measures the fraction of actual violations detected. F1 summarises their balance. The Bronze threshold was selected in an exploratory sweep; it is not claimed as a universal operational SLA.

The same grouped folds and leakage-safe feature exclusions are used in secondary regressions for 90th-percentile delay and jitter. These targets are evaluated on their native scales because the operational question concerns absolute milliseconds. Negative R^2 indicates that predictions generalise worse than the validation-fold mean.

C. Allocation Recommender

After grouped validation, a final XGBoost surrogate is fitted on all available rows. Five WFQ profiles are evaluated: 60/30/10, 50/40/10, 33/33/34, 25/65/10, and 80/15/5, where values denote Gold/Silver/Bronze queue weights. For each profile, the scheduler and weight features are changed,

TABLE I
FIVE-FOLD SIMULATION-GROUPED RESULTS

Class	MAE (ms)	R^2	Prec.	Recall	F1
Gold	3.17	0.907	0.986	0.898	0.940
Silver	3.67	0.898	0.798	0.593	0.680
Bronze	21.57	0.680	0.569	0.540	0.554

predicted delays are aggregated by class, and feasibility is checked against the same 30/50/60-ms mean-delay limits. Feasible profiles are ranked by a priority-weighted normalised delay score,

$$S = 3\frac{d_G}{30} + 2\frac{d_S}{50} + \frac{d_B}{60}, \quad (1)$$

where d_G , d_S , and d_B are predicted class means in milliseconds. This is an offline what-if analysis, not a causal guarantee: profile changes are not replayed through the simulator.

D. Reproducibility

The data-generation, preprocessing, grouped evaluation, plotting, and recommendation stages are implemented as separate Python scripts. Randomised estimators use seed 42, and every reported prediction metric is calculated from out-of-fold rows rather than from training predictions. The grouped evaluator also records the fold unit explicitly in its output artifact, reducing the risk that row-wise and run-wise results are mixed during reporting. The generated dataset, experiment scripts, and result tables are available in the project repository. A permanent archival identifier will be added to the camera-ready paper.

IV. RESULTS

A. Per-Class Delay and SLA Prediction

Table I reports the primary grouped results. Overall R^2 is 0.747 with a delay-scale MAE of 13.19 ms. Performance is strongly class-dependent. Gold and Silver are predicted accurately ($R^2 = 0.907$ and 0.898), whereas Bronze falls to 0.680 and its MAE rises to 21.57 ms. The pattern is operationally plausible: protected traffic is governed more strongly by stable topology and scheduler configuration, while best-effort delay depends on transient contention that the static inputs do not observe.

Gold SLA triggers retain very high precision (0.986), making false control actions comparatively rare for the highest-priority class. Silver and Bronze are less reliable, especially in recall. The Bronze F1 of 0.554 shows that a usable aggregate regressor does not automatically produce a strong violation detector for best-effort traffic.

B. Effect of Grouping by Simulation

Table II and Fig. 1 compare the grouped protocol with a shuffled row-wise split. Grouping reduces overall R^2 from 0.766 to 0.747. The largest class-level reduction is for Bronze (0.704 to 0.680); its SLA F1 decreases from 0.592 to 0.554. Gold remains stable. The result indicates modest rather than

TABLE III
EXPLORATORY ROW-WISE MODEL COMPARISON

Model	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
XGBoost	0.766	0.914	0.908	0.704
MLP	0.766	0.912	0.908	0.705
FT-Transformer	0.762	0.900	0.898	0.703

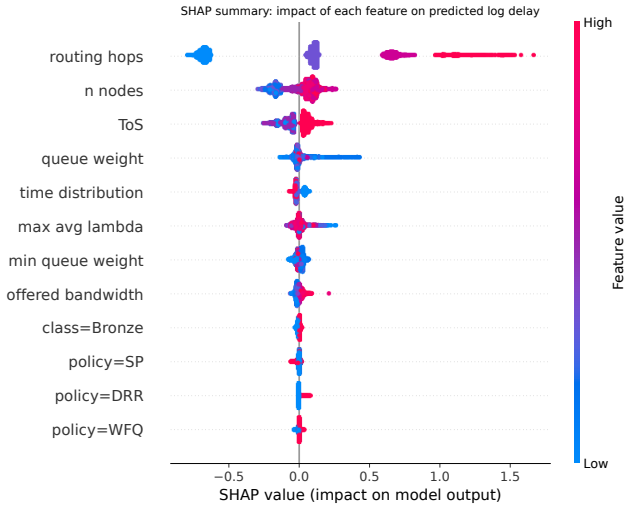


Fig. 2. SHAP summary for the XGBoost log-delay model; colour denotes feature value.

TABLE II
SENSITIVITY TO THE CROSS-VALIDATION UNIT

Protocol	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
Row K-fold	0.766	0.914	0.908	0.704
Grouped by run	0.747	0.907	0.898	0.680

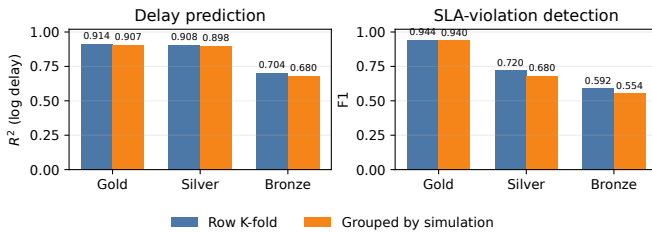


Fig. 1. Row-wise and simulation-grouped performance by QoS class.

catastrophic leakage, but it is large enough to affect claims about generalisation and therefore justifies simulation-level grouping.

An exploratory row-wise comparison produced similar global scores for XGBoost ($R^2 = 0.766$), a multilayer perceptron (0.766), and an FT-Transformer (0.762). Because these neural results have not yet been repeated with simulation-grouped outer folds, they are not treated as the paper's primary generalisation estimates. They nevertheless support

using XGBoost as the lower-cost surrogate for the allocation study.

C. Model Complexity and Feature Evidence

Table III expands the exploratory comparison by class. All three model families converge to essentially the same Bronze ceiling under the row-wise protocol. The FT-Transformer is marginally lower for Gold and Silver, while no difference exceeds 0.014 R^2 . This does not prove equivalence under grouped validation, but it gives no evidence that additional neural complexity compensates for the missing short-timescale traffic state.

Shapley additive explanations (SHAP) for the final XGBoost model provide a consistent explanation for this result (Fig. 2). Route length is the dominant feature: additional hops push predictions toward higher log delay, while shorter paths reduce it. Network size and traffic class follow. Queue weights and policy indicators have smaller global effects. These are associations within the simulator rather than causal estimates, but they show that the model relies primarily on stable path structure. Such structure is informative for protected Gold and Silver flows yet insufficient to describe the transient contention experienced by Bronze.

D. Tail Delay and Jitter

Table IV applies the grouped protocol to two secondary QoS targets. Tail delay is less predictable than average delay: overall R^2 is 0.293, with Gold and Silver reaching 0.550 and 0.490, while Bronze falls to 0.269. The corresponding Bronze MAE is 51.81 ms, consistent with rare contention events dominating the upper tail.

Jitter does not generalise from the static feature set. Overall R^2 is -0.040 , and every class is negative. Gold and Silver have small absolute MAE but extremely low within-class variance, so modest errors are worse than simply predicting the validation-fold mean. These results sharpen the observability boundary: configuration and path features describe mean delay, partially describe tail delay, and do not describe moment-to-moment delay variation.

E. WFQ Allocation Recommendation

Table V shows the offline what-if results. All five candidates satisfy the class mean-delay limits, but 60/30/10 obtains the lowest weighted score and is selected. The choice protects Gold while preserving sufficient Silver capacity; allocating 80% to Gold does not improve its predicted mean and raises Silver delay. The 33/33/34 profile increases Bronze mean delay despite assigning it a larger nominal weight, illustrating that the surrogate captures interactions rather than applying a simple inverse-weight rule.

The recommender is deliberately bounded. If no candidate meets the thresholds, it can return an infeasibility signal rather than silently applying the least-bad policy. A controller could then escalate to rerouting or capacity augmentation. In the present experiment all candidates are mean-feasible, so the ranking differentiates them by expected class-weighted delay.

TABLE IV
GROUPED RESULTS FOR SECONDARY QoS TARGETS

Target	Class	MAE (ms)	R^2
Delay p90	Overall	32.51	0.293
	Gold	8.92	0.550
	Silver	10.87	0.490
	Bronze	51.81	0.269
Jitter	Overall	3.22	-0.040
	Gold	0.51	-7.555
	Silver	1.01	-13.182
	Bronze	5.28	-0.034

TABLE V
PREDICTED MEAN DELAY UNDER CANDIDATE WFQ PROFILES

Profile	Gold (ms)	Silver (ms)	Bronze (ms)	Score
60/30/10	24.69	24.22	31.03	3.954
25/65/10	25.02	24.33	31.03	3.992
50/40/10	25.35	24.86	31.03	4.046
33/33/34	24.96	24.44	34.93	4.056
80/15/5	24.92	29.39	31.10	4.186

V. DISCUSSION AND LIMITATIONS

The central finding is not simply that delay is predictable; it is that predictability depends on traffic class, target, and validation design. Route length, network size, and class encode stable structure and explain protected mean delay well. Bronze is served after higher-priority traffic and is more sensitive to unobserved bursts and queue occupancy. The collapse on jitter confirms that increasing model complexity cannot recover information absent from the features.

Several limitations constrain the claims. First, the data are simulator-generated and cover a light-to-moderate load regime. Grouped folds test unseen runs from the same generator, not transfer to enterprise SD-WANs. Second, the feature set is static. Telemetry such as instantaneous queue occupancy, arrival-rate windows, and recent loss would be necessary to model transient behaviour. Third, the SLA thresholds are experimental operating points rather than contractual values. Fourth, the allocation recommender edits model inputs without rerunning the network, so it provides candidate screening rather than causal validation. Future work should repeat

every model comparison under grouped outer folds, replay recommended profiles in BNNetSimulator, evaluate heavier congestion, and validate on controller telemetry.

VI. CONCLUSION

This paper presented a simulation-grouped, per-class evaluation of QoS prediction for SD-WAN-like networks. XGBoost generalises well to unseen simulation runs for Gold and Silver mean delay but is less reliable for Bronze and does not generalise for jitter. Row-wise cross-validation overstates performance, confirming that the simulation run must be the evaluation unit. A WFQ recommender demonstrates how the validated model can rank candidate allocations while retaining explicit feasibility checks. The broader implication is that trustworthy AI-assisted QoS control requires class-aware metrics, group-safe validation, and honest bounds on what static network features can support.

REFERENCES

- [1] S. Rahman, T. Ahmed, M. Huynh, M. Tornatore, and B. Mukherjee, "Auto-scaling vnfs using machine learning to improve qos and reduce cost," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2018, pp. 1–6.
- [2] W. Zheng, M. Yang, C. Zhang, Y. Zheng, Y. Wu, Y. Zhang, and J. Li, "Application-aware qos routing in sdns using machine learning techniques," *Peer-to-Peer Networking and Applications*, vol. 15, no. 1, pp. 529–548, 2022.
- [3] L. Dinh, P. T. A. Quang, and J. Leguay, "Towards safe load balancing based on control barrier functions and deep reinforcement learning," *arXiv preprint arXiv:2401.05525*, 2024.
- [4] Y. Chen, A. Gu, L. Cui, and L. Lin, "MTEAL: Network routing optimization of SD-WAN traffic engineering integrating multi-dimensional QoS metrics," *Journal of Network and Computer Applications*, vol. 242, p. 104272, 2025.
- [5] P. T. A. Quang, X. Gong, C. Liu, K. Li, J. Leguay, X. Zhang, Y. Zhang, J. Li, and K. Ye, "Routing and qos policy optimization in sd-wan," *arXiv preprint arXiv:2209.12515*, 2022.
- [6] P. T. A. Quang, J. Leguay, X. Gong, and H. Xu, "Global qos policy optimization in sd-wan," in *Proc. IEEE 9th Int. Conf. Network Softwarization (NetSoft)*, 2023, pp. 202–206.
- [7] BNN-UPC, "BNNetSimulator: A packet-level network simulator to generate datasets for research and analysis," GitHub repository: <https://github.com/BNN-UPC/BNNetSimulator>, 2023.
- [8] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, "Routenet-fermi: Network modeling with graph neural networks," *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, 2023.
- [9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [10] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.